ИТОГОВЫЙ ОТЧЁТ

Защита капстоуна

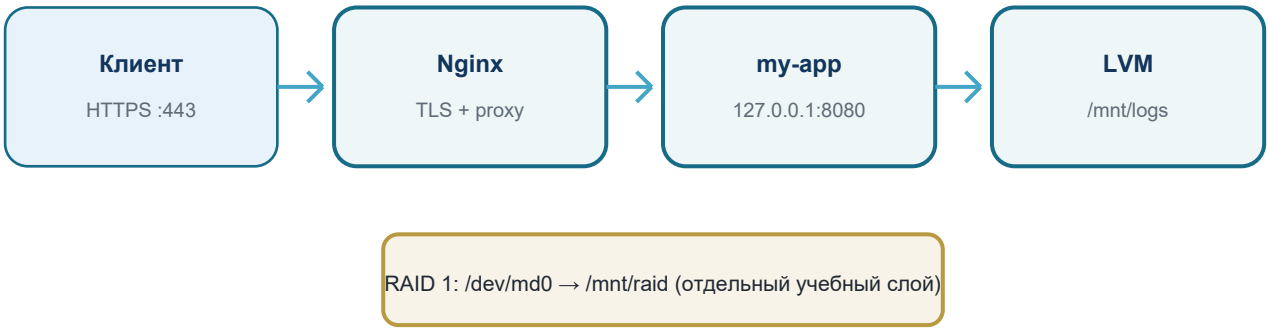
«Системное администрирование»

Студент	Ведина Юлия Алексеевна
Дорожка	Продвинутый
Репозиторий	https://github.com/JuliGalaxy/my-sysadmin-scripts
Преподаватель	Терещенко С. Н.
Среда	RED OS 7.3.5 · 10.0.19.113

2026

1. Краткое описание решения

На постоянном хосте RED OS 7.3.5 развёрнут единый воспроизводимый стенд. Скрипт варианта Б каждые 10 секунд записывает показатели памяти, дисков и нагрузки в monitor.log. Приложение работает в Docker-контейнере, управляется systemd и публикуется наружу только через Nginx с TLS. Команда sudo ./bootstrap.sh устанавливает зависимости и разворачивает все компоненты без интерактивного ввода.



Компонент	Роль	Результат
script.sh	Мониторинг ресурсов	monitor.log
Docker	Изолированный запуск	my-script:latest / my-app
LVM	Постоянное хранилище журнала	vg_data/lv_logs → /mnt/logs
RAID 1	Демонстрация зеркалирования	/dev/md0 → /mnt/raid
Nginx + TLS	Внешняя точка входа	HTTPS 200, HTTP → HTTPS
systemd	Автозапуск и перезапуск	enabled, active (running)

Итог	Стенд проверен end-to-end: HTTPS-запрос проходит через Nginx в контейнер, возвращает файл с LVM-тома и фиксируется в обоих уровнях журналирования.

01 Публичный Git-репозиторий

4 балла

Команда

```
git log --oneline -10
```

Результат

```
cf4f7a3 docs: добавлен отчёт проверки капстоуна
c3dd838 fix: отключён стандартный HTTP-сайт RED OS
d0517ff fix: создание каталогов для TLS-сертификата
2c78ccb fix: учтён установленный Docker CE в RED OS
58514e5 feat: добавлено развёртывание с Nginx TLS и systemd
0070ab0 feat: добавлена настройка RAID 1 и LVM
22d9443 feat: контейнеризован мониторинг ресурсов
48be7a3 docs: добавлен README и пример вывода
26cf431 fix: добавлены проверки и корректная остановка
cfbe2fc draft: первая версия скрипта
```

Что доказано Репозиторий содержит README и десять осмысленных коммитов. История показывает последовательное развитие решения: черновик, проверки, контейнеризация, хранилище, TLS/systemd и исправления под RED OS.

Публичная ссылка: https://github.com/JuliGalaxy/my-sysadmin-scripts

02 Скрипт автоматизации

4 балла

Команда

```
sudo docker run --rm --entrypoint /bin/bash my-script -c '
  MONITOR_ONCE=1 MONITOR_LOG=/tmp/monitor.log /usr/local/bin/script.sh;
  head -20 /tmp/monitor.log'
```

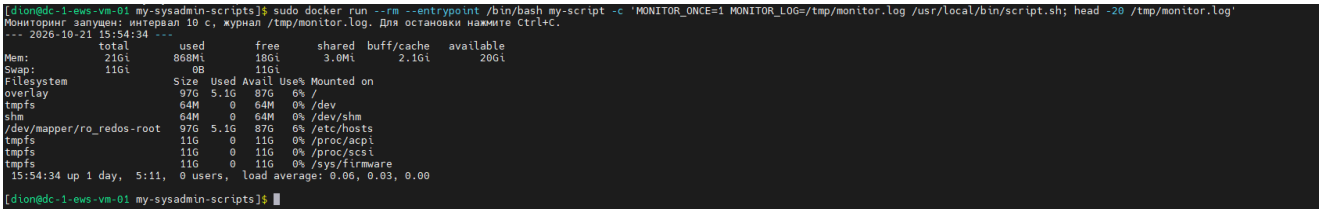


Рисунок 1 — Однократный контрольный запуск скрипта мониторинга внутри образа

Что доказано Скрипт варианта Б запускается без интерактивного ввода и формирует снимок с временной меткой, выводом free -h, df -h и uptime. Служебный режим MONITOR_ONCE используется только для воспроизводимой проверки.

03 Dockerfile, сборка и запуск контейнера

4 балла

Команды

```
sudo docker build --progress=plain -t my-script .
sudo docker ps --filter name=my-app
```

```
-bash: $'\E[200~sudo': command not found
[dion@dc-1-ews-vm-01 my-sysadmin-scripts]$ sudo docker build --progress=plain -t my-script .
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 627B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/ubuntu:22.04
#2 DONE 2.2s

#3 [internal] load .dockerignore
#3 transferring context: 61B done
#3 DONE 0.0s

#4 [1/5] FROM docker.io/library/ubuntu:22.04@sha256:0a9887931b0e38af2e3487ed39e29436b0535c4f3f52e57745baaf79d1898956
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 31B done
#5 DONE 0.0s

#6 [4/5] COPY script.sh /usr/local/bin/script.sh
#6 CACHED

#7 [2/5] RUN apt-get update     && apt-get install -y --no-install-recommends python3 ca-certificates     && rm -rf /var/lib/apt/lists/*
#7 CACHED

#8 [3/5] WORKDIR /var/www
#8 CACHED

#9 [5/5] RUN chmod +x /usr/local/bin/script.sh
#9 CACHED

#10 exporting to image
#10 exporting layers done
#10 writing image sha256:513aef6dacc1be10b3465f6acbec8ba6cf801c4ddffbec02c0c0bcf7089159ad done
#10 naming to docker.io/library/my-script done
#10 DONE 0.0s
[dion@dc-1-ews-vm-01 my-sysadmin-scripts]$
```

Рисунок 2 — Успешная сборка образа my-script:latest из Dockerfile

```
[dion@dc-1-ews-vm-01 my-sysadmin-scripts]$
[dion@dc-1-ews-vm-01 my-sysadmin-scripts]$ sudo docker ps --filter name=my-app
CONTAINER ID   IMAGE             COMMAND                  CREATED          STATUS                    PORTS                      NAMES
3a151ca22108   my-script:latest   "/bin/bash -c 'MONIT…"   25 minutes ago   Up 25 minutes (healthy)   127.0.0.1:8080->8080/tcp   my-app
[dion@dc-1-ews-vm-01 my-sysadmin-scripts]$
```

Рисунок 3 — Контейнер my-app работает в состоянии healthy

Что доказано Dockerfile успешно собран, а контейнер my-app работает и прошёл healthcheck. Порт 8080 опубликован только на 127.0.0.1, поэтому прямой внешний доступ к приложению закрыт.

04 RAID 1 и LVM

4 балла

Команды

```
cat /proc/mdstat
sudo lvs
df -h /mnt/raid /mnt/logs
```

```
[dion@dc-1-ews-vm-01 my-sysadmin-scripts]$ cat /proc/mdstat
Personalities : [raid1]
md0 : active raid1 loop1[1] loop0[0]
      523264 blocks super 1.2 [2/2] [UU]

unused devices: <none>
[dion@dc-1-ews-vm-01 my-sysadmin-scripts]$ sudo lvs
  LV      VG       Attr       LSize   Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  root    ro_redos -wi-ao----  98.89g
  swap    ro_redos -wi-ao----  11.10g
  lv_logs vg_data  -wi-ao---- 200.00m
[dion@dc-1-ews-vm-01 my-sysadmin-scripts]$ df -h /mnt/raid /mnt/logs
Filesystem                  Size  Used Avail Use% Mounted on
/dev/md0                    485M   14K  456M   1% /mnt/raid
/dev/mapper/vg_data-lv_logs  189M  205K  174M   1% /mnt/logs
[dion@dc-1-ews-vm-01 my-sysadmin-scripts]$
```

Рисунок 4 — Состояние RAID 1, логического тома и точек монтирования

Что доказано Массив /dev/md0 собран как RAID 1 из двух loop-устройств; состояние [UU] подтверждает доступность обоих участников. Отдельно создан LVM-том vg_data/lv_logs; файловые системы смонтированы в /mnt/raid и /mnt/logs.

05 Nginx как обратный прокси

4 балла

Команды

```
sudo nginx -t
curl -I http://10.0.19.113/monitor.log
```

```
[dion@dc-1-ews-vm-01 my-sysadmin-scripts]$
[dion@dc-1-ews-vm-01 my-sysadmin-scripts]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[dion@dc-1-ews-vm-01 my-sysadmin-scripts]$ curl -I http://10.0.19.113/monitor.log
HTTP/1.1 301 Moved Permanently
Server: nginx/1.30.4
Date: Wed, 21 Oct 2026 16:07:18 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://10.0.19.113/monitor.log
```

Рисунок 5 — Проверка конфигурации Nginx и перенаправление HTTP → HTTPS

Что доказано Проверка конфигурации завершилась сообщениями syntax is ok и test is successful. HTTP-запрос получает код 301 и перенаправляется на HTTPS; Nginx выступает единственной внешней точкой входа.

06 TLS и доступ по HTTPS

4 балла

Команда

```
curl -kI https://10.0.19.113/monitor.log
```



Рисунок 6 — Успешный HTTPS-запрос к опубликованному сервису

Что доказано	Ответ HTTP/1.1 200 OK подтверждает, что TLS-блок активен, сертификат подключён и Nginx успешно передаёт запрос контейнеру. Флаг -k ожидаем для самоподписанного сертификата учебного стенда.

07 Управление контейнером через systemd 3 балла

Команда

```
systemctl status my-app --no-pager
```

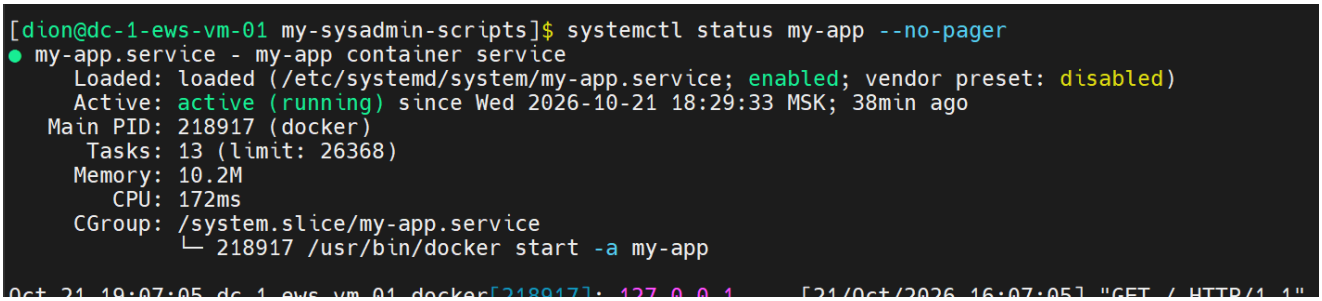


Рисунок 7 — Служба my-app включена и находится в состоянии active (running)

Что доказано	Юнит my-app.service загружен, включён в автозапуск (enabled) и активен. В cgroup службы виден процесс docker start -a my-app; зависимости Requires/After задают запуск после Docker.

Примечание: vendor preset: disabled описывает политику пакета по умолчанию и не отменяет фактическое состояние enabled.

08 Журналы systemd и Nginx

3 балла

Команды

```
sudo journalctl -u my-app -n 10 --no-pager
sudo tail -5 /var/log/nginx/access.log
```

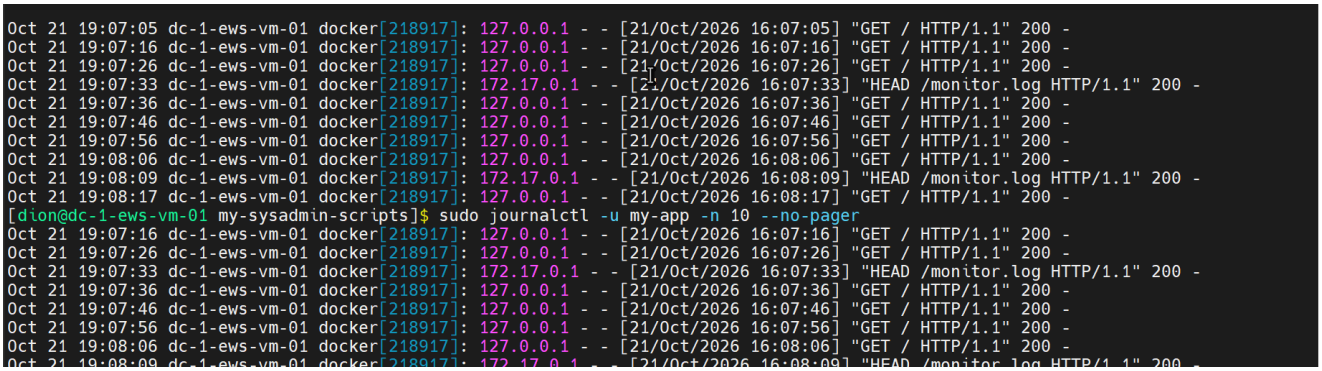


Рисунок 8 — Запросы к monitor.log в журнале службы my-app

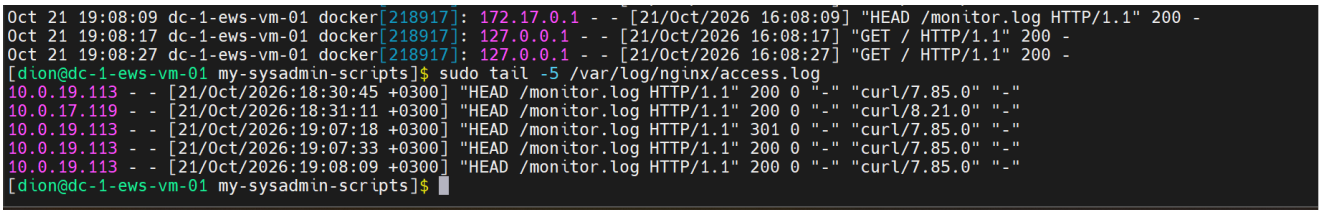


Рисунок 9 — Тот же контрольный запрос с кодом 200 в access.log Nginx

Что доказано После контрольного HTTPS-запроса обращение к /monitor.log с кодом 200 видно в journalctl и access.log. Это подтверждает полный маршрут запроса Nginx → контейнер и работоспособность обоих уровней журналирования.

2. Архитектурные решения

2.1 Почему выбран вариант Б

Выбран вариант Б — мониторинг ресурсов. Он формирует полезный непрерывно обновляемый артефакт monitor.log, не требует интерактивного ввода и естественно переносится в контейнер. Вариант A сильнее зависит от журналов и прав хостовой системы и потребовал бы отдельного read-only-монтирования. Вариант B предназначен для изменения учётных записей и хуже подходит для демонстрации постоянно работающего сетевого сервиса.

2.2 RAID 1 и LVM решают разные задачи

RAID 1 дублирует данные на двух устройствах и повышает отказоустойчивость ценой половины суммарной ёмкости. LVM создаёт абстракцию над блочными устройствами и упрощает управление логическими томами и их размером, но сам по себе данные не дублирует. В стенде они реализованы параллельно: /dev/md0 демонстрирует зеркалирование, а vg_data/lv_logs хранит журнал приложения. В промышленной схеме LVM можно разместить поверх RAID и совместить обе функции.

2.3 Постоянство данных и сетевой контур

Каталог /mnt/logs подключён в контейнер как /var/www. Поэтому monitor.log сохраняется независимо от пересоздания контейнера и одновременно доступен встроенному HTTP-серверу. Контейнер привязан к 127.0.0.1:8080: внешний клиент не обходит Nginx, а TLS, редирект и access.log сосредоточены в одном пограничном компоненте.

2.4 Почему self-signed сертификат

Самоподписанный RSA-сертификат выбран из-за закрытого IP-адреса и отсутствия публичного доменного имени. Let’s Encrypt потребовал бы общедоступный домен и подтверждение владения; внутренняя PKI избыточна для одного учебного хоста. В production следует использовать сертификат доверенного центра с корректным SAN и автоматическим продлением; self-signed и curl -k там неприемлемы.

3. Альтернативы, исправления и вывод

3.1 Рассмотренные альтернативы

Решение	Выбранный подход	Почему не альтернатива
Публикация порта	Только 127.0.0.1 + Nginx	0.0.0.0:8080 обошёл бы TLS и единое журналирование.
TLS	Self-signed для лаборатории	Let’s Encrypt невозможен без публичного домена; внутренняя PKI избыточна.
Запуск	systemd управляет контейнером	Ручной docker run не даёт системного автозапуска и единого статуса.
Хранилище	LVM bind-mount	Данные внутри writable layer исчезают при пересоздании контейнера.

3.2 Что не получилось с первого раза

• В RED OS уже был установлен Docker CE: bootstrap.sh изменён так, чтобы установка выполнялась только при отсутствии команды docker.

• Каталоги для ключа и сертификата могли отсутствовать: перед openssl добавлено их явное создание.

• Стандартный server-блок RED OS перехватывал HTTP-запросы: deploy.sh сохраняет резервную копию и отключает только конфликтующий демонстрационный блок.

Эти исправления отражены отдельными коммитами, поэтому Git-история показывает реальный процесс диагностики и доработки, а не единовременную загрузку готового результата.

3.3 Итоговая самопроверка

Пункт	Статус	Доказательство
1. Git	Выполнено	git log, README, public URL
2. Скрипт	Выполнено	контрольный запуск
3. Docker	Выполнено	build + healthy container
4. RAID/LVM	Выполнено	[UU], lvs, df -h
5. Nginx	Выполнено	nginx -t, HTTP 301
6. TLS	Выполнено	HTTPS 200
7. systemd	Выполнено	enabled, active
8. Логи	Выполнено	journalctl + access.log

Вывод	Все восемь пунктов технического чек-листа подтверждены свежим выводом команд. Репозиторий позволяет повторно развернуть стенд командой sudo ./bootstrap.sh и проверить маршрут запроса end-to-end.